



Fiche 1 :

`pthread_cleanup_push`

◆ Utilité :

- Permet d'enregistrer une fonction de nettoyage qui s'exécutera **automatiquement** si le thread est annulé.
- Évite les fuites de mémoire et les fichiers laissés ouverts en cas d'interruption brutale.

◆ Exemple concret :



Imagine un livreur qui transporte un colis fragile.

Si le livreur est forcé d'abandonner sa livraison (accident, panne), une procédure automatique est déclenchée pour récupérer et sécuriser le colis afin qu'il ne soit pas perdu.



`pthread_cleanup_pop`

◆ Utilité :

- Retire une fonction de nettoyage enregistrée avec `pthread_cleanup_push`.
- Peut **exécuter immédiatement** la fonction de nettoyage ou simplement l'enlever sans l'exécuter.

◆ Exemple concret :



Imagine que tu cuisines et que tu as mis une alarme pour te rappeler de nettoyer.

- Si tu finis de cuisiner et tu veux nettoyer immédiatement, tu laisses sonner l'alarme et tu nettoies →
(`pthread_cleanup_pop(1)`).
- Si tu décides finalement que le nettoyage n'est pas nécessaire, tu désactives juste l'alarme sans rien faire → (`pthread_cleanup_pop(0)`).

`pthread_cleanup_push`

• ◆ Utilité :

- Évite les fuites de mémoire et les fichiers laissés ouverts en cas d'interruption brutale.
- Permet d'enregistrer une fonction de nettoyage qui s'exécutera **automatiquement** si le thread est annulé.
- il n'y a pas de valeurs de retour c'est une macro



Fiche 3 : `nanosleep`

◆ Utilité :

- Permet de **mettre en pause un thread** pour une durée précise (en nanosecondes).

◆ Exemple concret :

👉 **Imagine un feu de circulation automatique :**

Il doit **attendre précisément 30 secondes** avant de passer du rouge au vert.

`nanosleep` permet d'assurer que l'attente est exacte, sans gaspiller inutilement les ressources du processeur.



Fiche 4 : `pthread_setcancelstate`

◆ Utilité :

- Permet de **bloquer ou autoriser** l'annulation d'un thread.
- Évite les interruptions à des moments critiques pour garantir l'intégrité des données.

◆ Exemple concret :

👉 **Imagine que tu écris un fichier important sur une clé USB :**

Si quelqu'un coupe l'alimentation en plein milieu, le fichier pourrait être corrompu.

`pthread_setcancelstate` permet de **désactiver l'annulation** pendant l'écriture, puis de **la réactiver une fois terminé**, assurant ainsi la sécurité des données.



Fiche 5 : `pthread_setcanceltype`

◆ **Utilité :**

- Définit **comment** un thread peut être annulé :
 - **Asynchrone** → Il peut être annulé **à tout moment** (risque d'incohérences).
 - **Différé** → Il est annulé **uniquement à des points sûrs** (meilleure stabilité).

◆ **Exemple concret :**

```
pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, &attr);  
  
pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, &attr);
```



Fiche 6: `pthread_testcancel`

◆ **Utilité :**

- Vérifie si une **demande d'annulation** a été faite pour le thread.
- Si une demande d'annulation est en attente, le thread est **immédiatement arrêté proprement**.
- Utile pour permettre l'annulation **à des moments contrôlés**, évitant ainsi les interruptions brutales.

◆ **Exemple concret :**

👉 **Imagine un programme qui télécharge un fichier volumineux :**

Plutôt que d'arrêter brutalement le téléchargement, on insère des **points de contrôle** avec `pthread_testcancel()`. Ainsi, si l'utilisateur annule, le thread peut s'arrêter **proprement** après avoir géré les parties déjà téléchargées et nettoyé les fichiers temporaires.



Fiche 7:

`pthread_attr_setschedpolicy`

◆ Utilité :

- Définit **comment** le système va **prioriser l'exécution des threads**.
- Différentes politiques influencent l'ordre d'exécution des threads en fonction de leur importance.

◆ Exemple concret :

👉 Imagine une usine avec un robot qui gère des machines en temps réel :

Certains threads doivent **toujours être exécutés en priorité**, par exemple ceux qui lisent des capteurs de sécurité. On utilise alors `SCHED_FIFO` pour que ces threads s'exécutent en **premier**, sans être interrompus par d'autres moins urgents.



Fiche 8:

`pthread_attr_setschedparam`

◆ Utilité :

- Définit la **priorité d'exécution** d'un thread.
- Plus la priorité est élevée, plus le thread sera exécuté rapidement par le processeur.

◆ Exemple concret :

👉 Imagine un jeu vidéo :

Le **thread qui gère l'affichage** doit être **prioritaire** pour garantir un rendu fluide.

Le **thread qui charge les textures** peut tourner en arrière-plan avec une **priorité plus basse**, car il n'est pas critique pour la fluidité du jeu.



Fiche 9: `pthread_mutex_init`

◆ **Utilité :**

- Initialise un **mutex** (verrou) pour **éviter les accès simultanés** à une ressource partagée.
- Empêche les **conditions de course** où plusieurs threads modifient une même donnée en même temps, causant des incohérences.

◆ **Exemple concret :**

👉 **Imagine une application bancaire :**

Si deux personnes essaient de **retirer de l'argent du même compte en même temps**, un **mutex** garantit que **seule une opération s'exécute à la fois**, évitant ainsi un solde incorrect.



Fiche 10 : `pthread_cond_init`

◆ **Utilité :**

- Initialise une **variable de condition** pour permettre la synchronisation entre threads.
- Permet à un thread d'**attendre un événement** et à un autre de **le signaler** quand il se produit.

◆ **Exemple concret :**

👉 **Imagine un système de messagerie :**

- **Les producteurs** (envoyeurs) ajoutent des messages dans un buffer.
- **Les consommateurs** (lecteurs) lisent ces messages.
- Si le buffer est **plein**, les producteurs doivent **attendre** avant d'ajouter de nouveaux messages.
- Si le buffer est **vide**, les consommateurs doivent **attendre** qu'un message arrive.

- `pthread_cond_init` permet de gérer cette attente efficacement.



Fiche 11 : `pthread_once`

◆ Utilité :

- Assure qu'une **fonction d'initialisation** est exécutée **une seule fois**, même si plusieurs threads l'appellent en même temps.
- Évite les **problèmes de concurrence** lors de l'initialisation de ressources partagées.

◆ Exemple concret :

👉 Imagine une bibliothèque logicielle qui accède à une base de données :

- Au premier appel, **la connexion est créée**.
- Tous les autres threads utilisent cette connexion sans la réinitialiser.
- `pthread_once` garantit que cette initialisation ne se fait **qu'une seule fois**, évitant des ouvertures multiples inutiles.